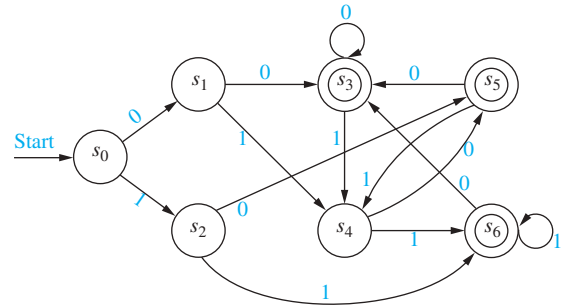


- *60. a) Show that s and t are 0-equivalent if and only if either both s and t are final states or neither s nor t is a final state. Conclude that each final state of $\bar{M}$, which is an R_* -equivalence class, contains only final states of M .
- b) Show that if k is a positive integer, then s and t are k -equivalent if and only if s and t are $(k-1)$ -equivalent and for every input symbol $a \in I$, $f(s, a)$ and $f(t, a)$ are $(k-1)$ -equivalent. Conclude that the transition function $\bar{f}$ is well-defined.
- c) Describe a procedure that can be used to construct the quotient automaton of a finite-automaton M .
- **61. a) Show that if M is a finite-state automaton, then the quotient automaton $\bar{M}$ recognizes the same language as M .
- b) Show that if M is a finite-state automaton with the property that for every state s of M there is a string $x \in I^*$ such that $f(s_0, x) = s$, then the quotient

automaton $\bar{M}$ has the minimum number of states of any finite-state automaton equivalent to M .

62. Answer these questions about the finite-state automaton M shown here.



- a) Find the k -equivalence classes of M for $k = 0, 1, 2$, and 3. Also, find the $*$ -equivalence classes of M .
- b) Construct the quotient automaton $\bar{M}$ of M .

13.4 Language Recognition

13.4.1 Introduction

We have seen that finite-state automata can be used as language recognizers. What sets can be recognized by these machines? Although this seems like an extremely difficult problem, there is a simple characterization of the sets that can be recognized by finite state automata. This problem was first solved in 1956 by the American mathematician Stephen Kleene. He showed that there is a finite-state automaton that recognizes a set if and only if this set can be built up from the null set, the empty string, and singleton strings by taking concatenations, unions, and Kleene closures, in arbitrary order. Sets that can be built up in this way are called **regular sets**. Regular grammars were defined in Section 13.1. Because of the terminology used, it is not surprising that there is a connection between regular sets, which are the sets recognized by finite-state automata, and regular grammars. In particular, a set is regular if and only if it is generated by a regular grammar.

Finally, there are sets that cannot be recognized by any finite-state automata. We will give an example of such a set. We will briefly discuss more powerful models of computation, such as pushdown automata and Turing machines, at the end of this section. The regular sets are those that can be formed using the operations of concatenation, union, and Kleene closure in arbitrary order, starting with the empty set, the set consisting of the empty string, and singleton sets. We will see that the regular sets are those that can be recognized using a finite-state automaton. To define regular sets we first need to define regular expressions.

Definition 1

The *regular expressions* over a set I are defined recursively by:

- the symbol $\emptyset$ is a regular expression;
- the symbol λ is a regular expression;
- the symbol x is a regular expression whenever $x \in I$;
- the symbols $(\mathbf{AB})$, $(\mathbf{A} \cup \mathbf{B})$, and $\mathbf{A}^*$ are regular expressions whenever $\mathbf{A}$ and $\mathbf{B}$ are regular expressions.

Each regular expression represents a set specified by these rules:

- $\emptyset$ represents the empty set, that is, the set with no strings;
- λ represents the set $\{\lambda\}$, which is the set containing the empty string;
- x represents the set $\{x\}$ containing the string with one symbol x ;
- $(\mathbf{AB})$ represents the concatenation of the sets represented by $\mathbf{A}$ and by $\mathbf{B}$;
- $(\mathbf{A} \cup \mathbf{B})$ represents the union of the sets represented by $\mathbf{A}$ and by $\mathbf{B}$;
- $\mathbf{A}^*$ represents the Kleene closure of the set represented by $\mathbf{A}$.

Sets represented by regular expressions are called **regular sets**. Henceforth regular expressions will be used to describe regular sets, so when we refer to the regular set $\mathbf{A}$, we will mean the regular set represented by the regular expression $\mathbf{A}$. Note that we will leave out outer parentheses from regular expressions when they are not needed.

Example 1 shows how regular expressions are used to specify regular sets.

EXAMPLE 1 What are the strings in the regular sets specified by the regular expressions 10^* , $(10)^*$, $0 \cup 01$, $0(0 \cup 1)^*$, and $(0^*1)^*$?

Solution: The regular sets represented by these expressions are given in Table 1, as the reader should verify. 

Finding a regular expression that specifies a given set can be quite tricky, as Example 2 illustrates.

EXAMPLE 2 Find a regular expression that specifies each of these sets:

- (a) the set of bit strings with even length
- (b) the set of bit strings ending with a 0 and not containing 11
- (c) the set of bit strings containing an odd number of 0s

Solution: (a) To construct a regular expression for the set of bit strings with even length, we use the fact that such a string can be obtained by concatenating zero or more strings each consisting of two bits. The set of strings of two bits is specified by the regular expression $(00 \cup 01 \cup 10 \cup 11)$. Consequently, the set of strings with even length is specified by $(00 \cup 01 \cup 10 \cup 11)^*$.

(b) A bit string ending with a 0 and not containing 11 must be the concatenation of one or more strings where each string is either a 0 or a 10. (To see this, note that such a bit string must consist of 0 bits or 1 bits each followed by a 0; the string cannot end with a single 1 because we know it ends with a 0.) It follows that the regular expression $(0 \cup 10)^*(0 \cup 10)$ specifies the set of bit strings that do not contain 11 and end with a 0. [Note that the set specified by $(0 \cup 10)^*$ includes the empty string, which is not in this set, because the empty string does not end with a 0.]

TABLE 1

<i>Expression</i>	<i>Strings</i>
10^*	a 1 followed by any number of 0s (including no zeros)
$(10)^*$	any number of copies of 10 (including the null string)
$0 \cup 01$	the string 0 or the string 01
$0(0 \cup 1)^*$	any string beginning with 0
$(0^*1)^*$	any string not ending with 0

(c) A bit string containing an odd number of 0s must contain at least one 0, which tells us that it starts with zero or more 1s, followed by a 0, followed by zero or more 1s. That is, each such bit string begins with a string of the form $1^j 0 1^k$ for nonnegative integers j and k . Because the bit string contains an odd number of 0s, additional bits after this initial block can be split into blocks each starting with a 0 and containing one more 0. Each such block is of the form $0 1^p 0 1^q$, where p and q are nonnegative integers. Consequently, the regular expression $1^* 0 1^* (0 1^* 0 1^*)^*$ specifies the set of bit strings with an odd number of 0s. ◀

13.4.2 Kleene's Theorem

In 1956 Kleene proved that regular sets are the sets that are recognized by a finite-state automaton. Consequently, this important result is called Kleene's theorem.

THEOREM 1

KLEENE'S THEOREM A set is regular if and only if it is recognized by a finite-state automaton.

Links

Kleene's theorem is one of the central results in automata theory. We will prove the *only if* part of this theorem, namely, that every regular set is recognized by a finite-state automaton. The proof of the *if* part, that a set recognized by a finite-state automaton is regular, is left as an exercise for the reader.

Proof: Recall that a regular set is defined in terms of regular expressions, which are defined recursively. We can prove that every regular set is recognized by a finite-state automaton if we can do the following things.



1. Show that $\emptyset$ is recognized by a finite-state automaton.
2. Show that $\{\lambda\}$ is recognized by a finite-state automaton.
3. Show that $\{a\}$ is recognized by a finite-state automaton whenever a is a symbol in I .
4. Show that AB is recognized by a finite-state automaton whenever both A and B are.
5. Show that $A \cup B$ is recognized by a finite-state automaton whenever both A and B are.
6. Show that A^* is recognized by a finite-state automaton whenever A is.

We now consider each of these tasks. First, we show that $\emptyset$ is recognized by a nondeterministic finite-state automaton. To do this, all we need is an automaton with no final states. Such an automaton is shown in Figure 1(a).

Second, we show that $\{\lambda\}$ is recognized by a finite-state automaton. To do this, all we need is an automaton that recognizes λ , the null string, but not any other string. This can be done by making the start state s_0 a final state and having no transitions, so that no other string takes s_0 to a final state. The nondeterministic automaton in Figure 1(b) shows such a machine.

Third, we show that $\{a\}$ is recognized by a nondeterministic finite-state automaton. To do this, we can use a machine with a starting state s_0 and a final state s_1 . We have a transition from s_0 to s_1 when the input is a , and no other transitions. The only string recognized by this machine is a . This machine is shown in Figure 1(c).

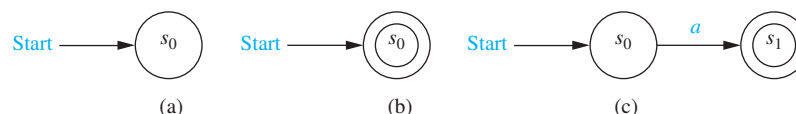


FIGURE 1 Nondeterministic finite-state automata that recognize some basic sets.

Next, we show that AB and $A \cup B$ can be recognized by finite-state automata if A and B are languages recognized by finite-state automata. Suppose that A is recognized by $M_A = (S_A, I, f_A, s_A, F_A)$ and B is recognized by $M_B = (S_B, I, f_B, s_B, F_B)$.

We begin by constructing a finite-state machine $M_{AB} = (S_{AB}, I, f_{AB}, s_{AB}, F_{AB})$ that recognizes AB , the concatenation of A and B . We build such a machine by combining the machines for A and B in series, so a string in A takes the combined machine from s_A , the start state of M_A , to s_B , the start state of M_B . A string in B should take the combined machine from s_B to a final state of the combined machine. Consequently, we make the following construction. Let S_{AB} be $S_A \cup S_B$. [Note that we can assume that S_A and S_B are disjoint.] The starting state s_{AB} is the same as s_A . The set of final states, F_{AB} , is the set of final states of M_B with s_{AB} included if and only if $\lambda \in A \cap B$. The transitions in M_{AB} include all transitions in M_A and in M_B , as well as some new transitions. For every transition in M_A that leads to a final state, we form a transition in M_{AB} from the same state to s_B , on the same input. In this way, a string in A takes M_{AB} from s_{AB} to s_B , and then a string in B takes s_B to a final state of M_{AB} . Moreover, for every transition from s_B we form a transition in M_{AB} from s_{AB} to the same state. Figure 2(a) contains an illustration of this construction.

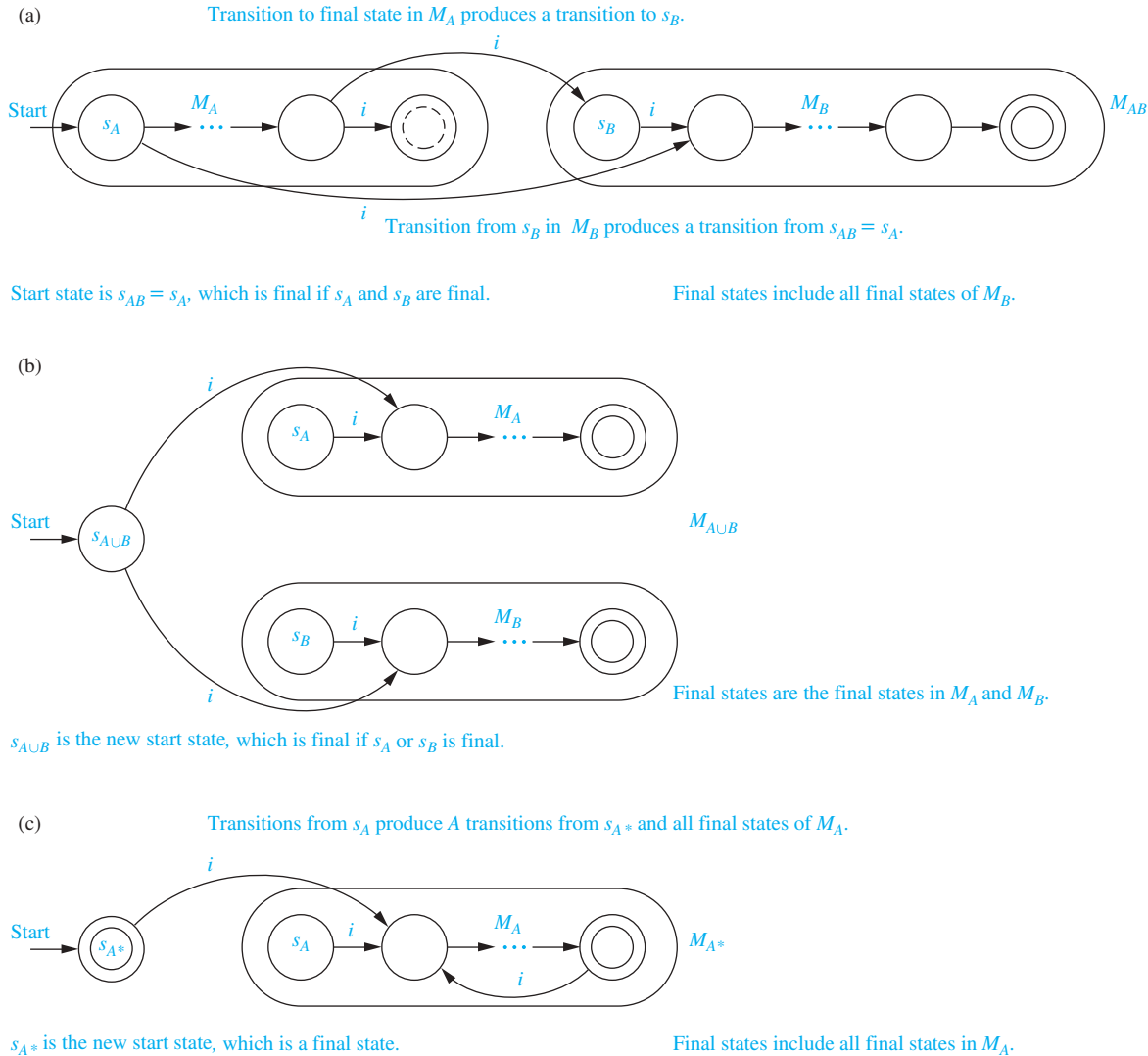


FIGURE 2 Building automata to recognize concatenations, unions, and Kleene closures.

We now construct a machine $M_{A \cup B} = (S_{A \cup B}, I, f_{A \cup B}, s_{A \cup B}, F_{A \cup B})$ that recognizes $A \cup B$. This automaton can be constructed by combining M_A and M_B in parallel, using a new start state that has the transitions that both s_A and s_B have. Let $S_{A \cup B} = S_A \cup S_B \cup \{s_{A \cup B}\}$, where $s_{A \cup B}$ is a new state that is the start state of $M_{A \cup B}$. Let the set of final states $F_{A \cup B}$ be $F_A \cup F_B \cup \{s_{A \cup B}\}$ if $\lambda \in A \cup B$, and $F_A \cup F_B$ otherwise. The transitions in $M_{A \cup B}$ include all those in M_A and in M_B . Also, for each transition from s_A to a state s on input i we include a transition from $s_{A \cup B}$ to s on input i , and for each transition from s_B to a state s on input i we include a transition from $s_{A \cup B}$ to s on input i . In this way, a string in A leads from $s_{A \cup B}$ to a final state in the new machine, and a string in B leads from $s_{A \cup B}$ to a final state in the new machine. Figure 2(b) illustrates the construction of $M_{A \cup B}$.

Finally, we construct $M_{A^*} = (S_{A^*}, I, f_{A^*}, s_{A^*}, F_{A^*})$, a machine that recognizes A^* , the Kleene closure of A . Let S_{A^*} include all states in S_A and one additional state s_{A^*} , which is the starting state for the new machine. The set of final states F_{A^*} includes all states in F_A as well as the start state s_{A^*} , because λ must be recognized. To recognize concatenations of arbitrarily many strings from A , we include all the transitions in M_A , as well as transitions from s_{A^*} that match the transitions from s_A , and transitions from each final state that match the transitions from s_A . With this set of transitions, a string made up of concatenations of strings from A will take s_{A^*} to a final state when the first string in A has been read, returning to a final state when the second string in A has been read, and so on. Figure 2(c) illustrates the construction we used. $\triangleleft$

A nondeterministic finite-state automaton can be constructed for any regular set using the procedure described in this proof. We illustrate how this is done with Example 3.

EXAMPLE 3 Construct a nondeterministic finite-state automaton that recognizes the regular set $1^* \cup 01$.

Solution: We begin by building a machine that recognizes 1^* . This is done using the machine that recognizes 1 and then using the construction for M_{A^*} described in the proof. Next, we build a machine that recognizes 01 , using machines that recognize 0 and 1 and the construction in the proof for M_{AB} . Finally, using the construction in the proof for $M_{A \cup B}$, we construct the machine for $1^* \cup 01$. The finite-state automata used in this construction are shown in Figure 3. The states in the successive machines have been labeled using different subscripts, even when a state is formed from one previously used in another machine. Note that the construction given here does not produce the simplest machine that recognizes $1^* \cup 01$. A much simpler machine that recognizes this set is shown in Figure 3(b). $\triangleleft$

13.4.3 Regular Sets and Regular Grammars

In Section 13.1 we introduced phrase-structure grammars and defined different types of grammars. In particular we defined regular, or type 3, grammars, which are grammars of the form $G = (V, T, S, P)$, where each production is of the form $S \rightarrow \lambda$, $A \rightarrow a$, or $A \rightarrow aB$, where a is a terminal symbol, and A and B are nonterminal symbols. As the terminology suggests, there is a close connection between regular grammars and regular sets.

THEOREM 2 A set is generated by a regular grammar if and only if it is a regular set.

Proof: First we show that a set generated by a regular grammar is a regular set. Suppose that $G = (V, T, S, P)$ is a regular grammar generating the set $L(G)$. To show that $L(G)$ is regular we will build a nondeterministic finite-state machine $M = (S, I, f, s_0, F)$ that recognizes $L(G)$. Let S , the set of states, contain a state s_A for each nonterminal symbol A of G and an additional state s_F , which is a final state. The start state s_0 is the state formed from the start symbol S . The transitions of M are formed from the productions of G in the following way. A transition

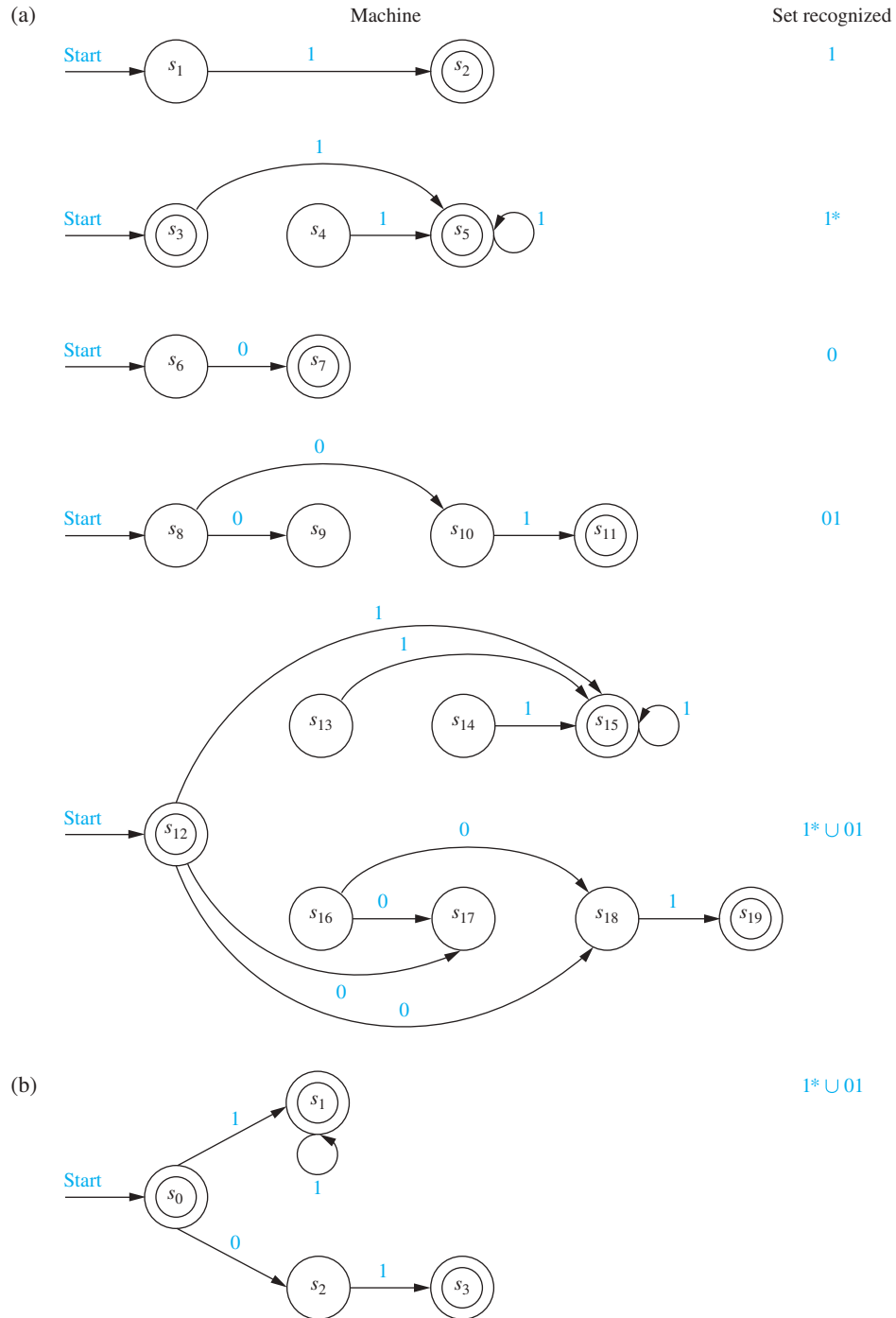
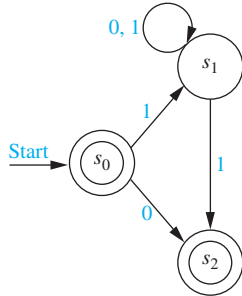


FIGURE 3 Nondeterministic finite-state automata recognizing $1^* \cup 01$.

from s_A to s_F on input of a is included if $A \rightarrow a$ is a production, and a transition from s_A to s_B on input of a is included if $A \rightarrow aB$ is a production. The set of final states includes s_F and also includes s_0 if $S \rightarrow \lambda$ is a production in G . It is not hard to show that the language recognized by M equals the language generated by the grammar G , that is, $L(M) = L(G)$. This can be done by determining the words that lead to a final state. The details are left as an exercise for the reader. ◀

Before giving the proof of the converse, we illustrate how a nondeterministic machine is constructed that recognizes the same set as a regular grammar.

EXAMPLE 4 Construct a nondeterministic finite-state automaton that recognizes the language generated by the regular grammar $G = (V, T, S, P)$, where $V = \{0, 1, A, S\}$, $T = \{0, 1\}$, and the productions in P are $S \rightarrow 1A$, $S \rightarrow 0$, $S \rightarrow \lambda$, $A \rightarrow 0A$, $A \rightarrow 1A$, and $A \rightarrow 1$.



Solution: The state diagram for a nondeterministic finite-state automaton that recognizes $L(G)$ is shown in Figure 4. This automaton is constructed following the procedure described in the proof. In this automaton, s_0 is the state corresponding to S , s_1 is the state corresponding to A , and s_2 is the final state. ◀

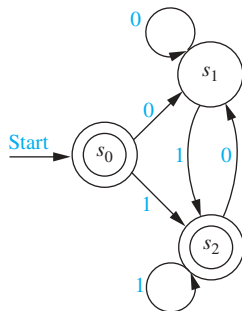
We now complete the proof of Theorem 2.

FIGURE 4 A nondeterministic finite-state automaton recognizing $L(G)$.

Proof: We now show that if a set is regular, then there is a regular grammar that generates it. Suppose that M is a finite-state machine that recognizes this set with the property that s_0 , the starting state of M , is never the next state for a transition. (We can find such a machine by Exercise 20.) The grammar $G = (V, T, S, P)$ is defined as follows. The set V of symbols of G is formed by assigning a symbol to each state of S and each input symbol in I . The set T of terminal symbols of G is the set I . The start symbol S is the symbol formed from the start state s_0 . The set P of productions in G is formed from the transitions in M . In particular, if the state s goes to a final state under input a , then the production $A_s \rightarrow a$ is included in P , where A_s is the nonterminal symbol formed from the state s . If the state s goes to the state t on input a , then the production $A_s \rightarrow aA_t$ is included in P . The production $S \rightarrow \lambda$ is included in P if and only if $\lambda \in L(M)$. Because the productions of G correspond to the transitions of M and the productions leading to terminals correspond to transitions to final states, it is not hard to show that $L(G) = L(M)$. We leave the details as an exercise for the reader. ◀

Example 5 illustrates the construction used to produce a grammar from an automaton that generates the language recognized by this automaton.

EXAMPLE 5 Find a regular grammar that generates the regular set recognized by the finite-state automaton shown in Figure 5.



Solution: The grammar $G = (V, T, S, P)$ generates the set recognized by this automaton where $V = \{S, A, B, 0, 1\}$, the symbols S, A , and B correspond to the states s_0, s_1 , and s_2 , respectively, $T = \{0, 1\}$, S is the start symbol; and the productions are $S \rightarrow 0A$, $S \rightarrow 1B$, $S \rightarrow \lambda$, $A \rightarrow 0A$, $A \rightarrow 1B$, $A \rightarrow 1$, $B \rightarrow 0A$, $B \rightarrow 1B$, and $B \rightarrow 1$. ◀

13.4.4 A Set Not Recognized by a Finite-State Automaton

We have seen that a set is recognized by a finite state automaton if and only if it is regular. We will now show that there are sets that are not regular by describing one such set. The technique used to show that this set is not regular illustrates an important method for showing that certain sets are not regular.

EXAMPLE 6 Show that the set $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$, made up of all strings consisting of a block of 0s followed by a block of an equal number of 1s, is not regular.

Solution: Suppose that this set were regular. Then there would be a nondeterministic finite-state automaton $M = (S, I, f, s_0, F)$ recognizing it. Let N be the number of states in this machine,



13.4.5 More Powerful Types of Machines



Finite-state automata are unable to carry out many computations. The main limitation of these machines is their finite amount of memory. This prevents them from recognizing languages that are not regular, such as $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$. Because a set is regular if and only if it is the language generated by a regular grammar, Example 6 shows that there is no regular grammar that generates the set $\{0^n 1^n \mid n = 0, 1, 2, \dots\}$. However, there is a context-free grammar that generates this set. Such a grammar was given in Example 5 in Section 13.1.

Because of the limitations of finite-state machines, it is necessary to use other, more powerful, models of computation. One such model is the **pushdown automaton**. A pushdown automaton includes everything in a finite-state automaton, as well as a stack, which provides unlimited memory. Symbols can be placed on the top or taken off the top of the stack. A set is recognized in one of two ways by a pushdown automaton. First, a set is recognized if the set consists of all the strings that produce an empty stack when they are used as input. Second, a set is recognized if it consists of all the strings that lead to a final state when used as input. It can be shown that a set is recognized by a pushdown automaton if and only if it is the language generated by a context-free grammar.

However, there are sets that cannot be expressed as the language generated by a context-free grammar. One such set is $\{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$. We will indicate why this set cannot be recognized by a pushdown automaton, but we will not give a proof, because we have not developed the machinery needed. (However, one method of proof is given in Exercise 28 of the supplementary exercises at the end of this chapter.) The stack can be used to show that a string begins with a sequence of 0s followed by an equal number of 1s by placing a symbol on the stack for each 0 (as long as only 0s are read), and removing one of these symbols for each 1 (as long as only 1s following the 0s are read). But once this is done, the stack is empty, and there is no way to determine that there are the same number of 2s in the string as 0s.

There are other machines called **linear bounded automata**, more powerful than pushdown automata, that can recognize sets such as $\{0^n 1^n 2^n \mid n = 0, 1, 2, \dots\}$. In particular, linear bounded automata can recognize context-sensitive languages. However, these machines cannot recognize all the languages generated by phrase-structure grammars. To avoid the limitations of special types of machines, the model known as a **Turing machine**, named after the British mathematician Alan Turing, is used. A Turing machine is made up of everything included in a finite-state machine together with a tape, which is infinite in both directions. A Turing machine has read and write capabilities on the tape, and it can move back and forth along this tape. Turing machines can recognize all languages generated by phrase-structure grammars. In addition, Turing machines can model all the computations that can be performed on a computing machine. Because of their power, Turing machines are extensively studied in theoretical computer science. We will briefly study them in Section 13.5.

Alan Turing invented Turing machines before modern computers existed!

Exercises

- Describe in words the strings in each of these regular sets.

a) 1^*0	b) 1^*00^*	e) $(01)^*(11)^*$	f) $0^*(10 \cup 11)^*$
c) $111 \cup 001$	d) $(1 \cup 00)^*$	g) $0^*(10)^*11$	h) $01(01 \cup 0)1^*$
e) $(00^*1)^*$	f) $(0 \cup 1)(0 \cup 1)^*00$		
- Describe in words the strings in each of these regular sets.

a) 001^*	b) $(01)^*$	a) 10^*1^*	b) $0^*(10 \cup 11)^*$
c) $01 \cup 001^*$	d) $0(11 \cup 0)^*$	c) $1(01)^*1^*$	d) $1^*01(0 \cup 1)$
e) $(101^*)^*$	f) $(0^* \cup 1)11$	e) $(10)^*(11)^*$	f) $1(00)^*(11)^*$
		g) $(10)^*1011$	h) $(1 \cup 00)(01 \cup 0)1^*$
- Determine whether 0101 belongs to each of these regular sets.

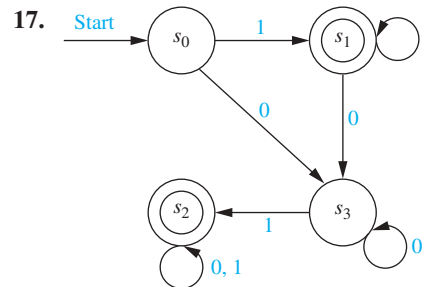
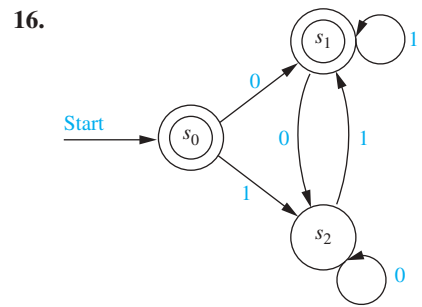
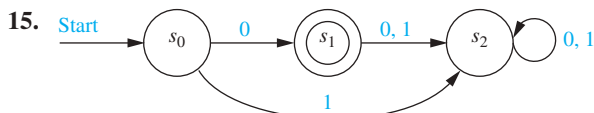
a) 01^*0^*	b) $0(11)^*(01)^*$
c) $0(10)^*1^*$	d) $0^*10(0 \cup 1)$
- Determine whether 1011 belongs to each of these regular sets.

a) 10^*1^*	b) $0^*(10 \cup 11)^*$
c) $1(01)^*1^*$	d) $1^*01(0 \cup 1)$
e) $(10)^*(11)^*$	f) $1(00)^*(11)^*$
g) $(10)^*1011$	h) $(1 \cup 00)(01 \cup 0)1^*$
- Express each of these sets using a regular expression.

a) the set consisting of the strings 0, 11, and 010
b) the set of strings of three 0s followed by two or more 0s
c) the set of strings of odd length

- d) the set of strings that contain exactly one 1
 e) the set of strings ending in 1 and not containing 000
6. Express each of these sets using a regular expression.
- a) the set containing all strings with zero, one, or two bits
 b) the set of strings of two 0s, followed by zero or more 1s, and ending with a 0
 c) the set of strings with every 1 followed by two 0s
 d) the set of strings ending in 00 and not containing 11
 e) the set of strings containing an even number of 1s
7. Express each of these sets using a regular expression.
- a) the set of strings of one or more 0s followed by a 1
 b) the set of strings of two or more symbols followed by three or more 0s
 c) the set of strings with either no 1 preceding a 0 or no 0 preceding a 1
 d) the set of strings containing a string of 1s such that the number of 1s equals 2 modulo 3, followed by an even number of 0s
8. Construct deterministic finite-state automata that recognize each of these sets from I^* , where I is an alphabet.
- a) $\emptyset$ b) $\{\lambda\}$ c) $\{a\}$, where $a \in I$
9. Construct nondeterministic finite-state automata that recognize each of the sets in Exercise 8.
10. Construct nondeterministic finite-state automata that recognize each of these sets.
- a) $\{\lambda, 0\}$ b) $\{0, 11\}$ c) $\{0, 11, 000\}$
- *11. Show that if A is a regular set, then A^R , the set of all reversals of strings in A , is also regular.
12. Using the constructions described in the proof of Kleene's theorem, find nondeterministic finite-state automata that recognize each of these sets.
- a) 01^* b) $(0 \cup 1)1^*$ c) $00(1^* \cup 10)$
13. Using the constructions described in the proof of Kleene's theorem, find nondeterministic finite-state automata that recognize each of these sets.
- a) 0^*1^* b) $(0 \cup 11)^*$ c) $01^* \cup 00^*1$
14. Construct a nondeterministic finite-state automaton that recognizes the language generated by the regular grammar $G = (V, T, S, P)$, where $V = \{0, 1, S, A, B\}$, $T = \{0, 1\}$, S is the start symbol, and the set of productions is
- a) $S \rightarrow 0A, S \rightarrow 1B, A \rightarrow 0, B \rightarrow 0$.
 b) $S \rightarrow 1A, S \rightarrow 0, S \rightarrow \lambda, A \rightarrow 0B, B \rightarrow 1B, B \rightarrow 1$.
 c) $S \rightarrow 1B, S \rightarrow 0, A \rightarrow 1A, A \rightarrow 0B, A \rightarrow 1, A \rightarrow 0, B \rightarrow 1$.

In Exercises 15–17 construct a regular grammar $G = (V, T, S, P)$ that generates the language recognized by the given finite-state machine.



18. Show that the finite-state automaton constructed from a regular grammar in the proof of Theorem 2 recognizes the set generated by this grammar.
19. Show that the regular grammar constructed from a finite-state automaton in the proof of Theorem 2 generates the set recognized by this automaton.
20. Show that every nondeterministic finite-state automaton is equivalent to another such automaton that has the property that its starting state is never revisited.
- *21. Let $M = (S, I, f, s_0, F)$ be a deterministic finite-state automaton. Show that the language recognized by M , $L(M)$, is infinite if and only if there is a word x recognized by M with $l(x) \geq |S|$.
- *22. One important technique used to prove that certain sets are not regular is the **pumping lemma**. The pumping lemma states that if $M = (S, I, f, s_0, F)$ is a deterministic finite-state automaton and if x is a string in $L(M)$, the language recognized by M , with $l(x) \geq |S|$, then there are strings u, v , and w in I^* such that $x = uvw$, $l(uv) \leq |S|$ and $l(v) \geq 1$, and $uv^i w \in L(M)$ for $i = 0, 1, 2, \dots$. Prove the pumping lemma. [Hint: Use the same idea as was used in Example 5.]
- *23. Show that the set $\{0^{2^n}1^n \mid n = 0, 1, 2, \dots\}$ is not regular using the pumping lemma given in Exercise 22.
- *24. Show that the set $\{1^{n^2} \mid n = 0, 1, 2, \dots\}$ is not regular using the pumping lemma from Exercise 22.
- *25. Show that the set of palindromes over $\{0, 1\}$ is not regular using the pumping lemma given in Exercise 22. [Hint: Consider strings of the form $0^N 10^N$.]
- **26. Show that a set recognized by a finite-state automaton is regular. (This is the *if* part of Kleene's theorem.)

Links ➤